تاسك: Conditionals & Loops

٥ سيناريوهات من الحياة الحقيقية — اكتب الكود بـ JavaScript وشغّله في الـ Console

`if / else` `switch` `for` `while` `do...while`

قبل ما تبدأ

- كل تاسك فيه **سيناريو** و**مطلوب** و**كود بداية** تكمّل عليه، وفي الآخر **الناتج المتوقع** عشان تقارن بيه.
- جرّب الكود بأكتر من قيمة — غيّر المتغيرات اللي فوق وشوف النتيجة بتتغير إزاي.
- التاسكات متدرّجة: من ١ لـ ٤ للمبتدئين، والتاسك ٥ أصعب شوية لأنه بيجمع أكتر من مفهوم.

١ نتيجة الامتحان

`if / else if / else` سهل

المدرسة عايزة برنامج صغير: المدرّس يكتب درجة الطالب من ١٠٠، والبرنامج يطبعله التقدير بتاعه بدل ما يحسبه بإيده لكل طالب.

المطلوب

1. لو الدرجة أقل من `0` أو أكبر من `100` اطبع: **درجة غير صحيحة**
2. من `85` لـ `100` ← **ممتاز**
3. من `75` لـ `84` ← **جيد جداً**
4. من `65` لـ `74` ← **جيد**
5. من `50` لـ `64` ← **مقبول**
6. أقل من `50` ← **راسب**

كود البداية

```
let score = 92;  // جرّب كمان: 70 و 40 و 120

//  اكتب الشروط هنا
```

الناتج المتوقع

```
score = 92   →  ممتاز
score = 70   →  جيد
score = 40   →  راسب
score = 120  →  درجة غير صحيحة
```

تلميح: ابدأ بشرط الدرجة الغلط الأول، واستخدم `||` عشان تجمع الحالتين في شرط واحد. وبعدها رتّب الشروط من الأكبر للأصغر.

٢ ماكينة المشروبات

`switch` سهل

في ماكينة مشروبات في الكافيتيريا. كل مشروب ليه رقم، والزبون بيدوس على الرقم والماكينة بتقوله اختار إيه وسعره كام.

المطلوب

الرقم	المشروب	السعر
1	قهوة	15 جنيه
2	شاي	10 جنيه
3	عصير برتقال	20 جنيه
4	مياه	5 جنيه

1. استخدم `switch` على متغير `drinkCode` واطبع: **طلبك: [اسم المشروب] - السعر: [السعر] جنيه**
2. لو الرقم مش موجود في الجدول اطبع: **اختيار غير موجود**

كود البداية

```
let drinkCode = 3; // جرّب كمان: 1 و 9

switch (drinkCode) {
  // اكتب الـ cases هنا
}
```

الناتج المتوقع

```
drinkCode = 3   →   طلبك: عصير برتقال - السعر: 20 جنيه
drinkCode = 1   →   طلبك: قهوة - السعر: 15 جنيه
drinkCode = 9   →   اختيار غير موجود
```

تلميح: متنساش `break` في آخر كل `case` — جرّب تشيلها من case واحدة وشوف إيه اللي هيحصل!

٣ برنامج الجري

`for` سهل

مدرّب الجيم عمل لك برنامج جري لمدة ٧ أيام: أول يوم تجري **1000 متر**، وكل يوم تزوّد **500 متر** عن اليوم اللي قبله.

المطلوب

1. استخدم `for` عشان تطبع مسافة كل يوم من اليوم ١ لليوم ٧.
2. لو المسافة في اليوم ده `3000` متر أو أكتر، اكتب جنبها **(يوم صعب)**.
3. في الآخر اطبع **إجمالي المسافة** اللي جريتها في الأسبوع.

كود البداية

```
let distance = 1000;
let total = 0;

for (/* ابدأ من يوم 1 لحد يوم 7 */) {
  // اطبع مسافة اليوم + زوّد الإجمالي + زوّد المسافة
}

// اطبع الإجمالي
```

الناتج المتوقع

```
اليوم 1: 1000 متر
اليوم 2: 1500 متر
اليوم 3: 2000 متر
اليوم 4: 2500 متر
اليوم 5: 3000 متر (يوم صعب)
اليوم 6: 3500 متر (يوم صعب)
اليوم 7: 4000 متر (يوم صعب)
إجمالي المسافة: 17500 متر
```

تلميح: رتّب الخطوات جوه الـ loop كويس: الأول اطبع، بعدين زوّد `total`، وفي الآخر زوّد `distance` بـ 500.

٤ الحصّالة

`while` متوسط

نفسك تشتري سماعة سعرها **1000 جنيه**. معاك **0 جنيه**، وكل أسبوع بتحط في الحصّالة **150 جنيه**. مش عارف هتاخد كام أسبوع — وده بالظبط وقت `while` .

المطلوب

1. طول ما الفلوس اللي معاك أقل من سعر السماعة، حط 150 جنيه وزوّد عدد الأسابيع.
2. اطبع كل أسبوع: **الأسبوع [رقم]: معاك [المبلغ] جنيه**
3. بعد ما الـ loop يخلص اطبع: **وصلت للهدف بعد [عدد] أسابيع، ومعاك [المبلغ] جنيه**

كود البداية

```
let price = 1000;
let savings = 0;
let weeks = 0;

while (/* الشرط هنا */) {
  // ...
}
```

الناتج المتوقع

```
الأسبوع 1: معاك 150 جنيه
الأسبوع 2: معاك 300 جنيه
الأسبوع 3: معاك 450 جنيه
الأسبوع 4: معاك 600 جنيه
الأسبوع 5: معاك 750 جنيه
الأسبوع 6: معاك 900 جنيه
الأسبوع 7: معاك 1050 جنيه
وصلت للهدف بعد 7 أسابيع، ومعاك 1050 جنيه
```

سؤال للتفكير: ليه استخدمنا `while` هنا مش `for` ؟ وإيه اللي هيحصل لو نسيت تزوّد `savings` جوه الـ loop؟

٥ ماكينة الـ ATM

`do...while + switch + if` أصعب شوية

بتعمل برنامج لماكينة ATM. الماكينة لازم تطلب الرقم السري **مرة واحدة على الأقل** (عشان كده `do...while`)، والمستخدم عنده **3 محاولات بس**. لو دخل صح يقدر يعمل عملية، ولو غلط ٣ مرات الكارت يتوقف.

البيانات

```
const correctPin = "1234";
let balance = 2000;

// المحاولات اللي المستخدم هيكتبها بالترتيب (بدل prompt)
let pinAttempts = ["1111", "2580", "1234"];

let operation = "withdraw"; // "balance" | "withdraw" | "deposit"
let amount = 500;
```

المطلوب

1. **الجزء الأول (do...while):** خد المحاولة اللي عليها الدور من `pinAttempts` وزوّد عدّاد المحاولات. لو غلط اطبع **رقم سري غلط، فاضلك [عدد] محاولات**. كرّر طول ما الرقم غلط و المحاولات أقل من 3.
2. بعد الـ loop: لو الرقم صح اطبع **أهلاً بيك**، ولو لأ اطبع **تم إيقاف الكارت** ووقّف البرنامج.
3. **الجزء التاني (switch):** نفّذ العملية حسب `operation`:
   - `"balance"` ← اطبع **رصيدك: [الرصيد] جنيه**
   - `"withdraw"` ← لو `amount` أكبر من الرصيد اطبع **رصيدك لا يكفي**، غير كده اخصم واطبع **تم سحب [المبلغ] جنيه، رصيدك الحالي: [الرصيد] جنيه**
   - `"deposit"` ← زوّد واطبع **تم إيداع [المبلغ] جنيه، رصيدك الحالي: [الرصيد] جنيه**
   - أي حاجة تانية ← **عملية غير معروفة**

الناتج المتوقع — الحالة ١ (البيانات زي ما هي)

```
رقم سري غلط، فاضلك 2 محاولات
رقم سري غلط، فاضلك 1 محاولات
أهلاً بيك
تم سحب 500 جنيه، رصيدك الحالي: 1500 جنيه
```

الناتج المتوقع — الحالة ٢

غيّر المحاولات لـ `["0000", "1111", "2222"]`:

```
رقم سري غلط، فاضلك 2 محاولات
رقم سري غلط، فاضلك 1 محاولات
رقم سري غلط، فاضلك 0 محاولات
تم إيقاف الكارت
```

الناتج المتوقع — الحالة ٣

رجّع المحاولات زي الأول، وخلّي `amount = 5000` :

```
رقم سري غلط، فاضلك 2 محاولات
رقم سري غلط، فاضلك 1 محاولات
أهلاً بيك
رصيدك لا يكفي
```

تلميح: اعمل متغير `attempts = 0` واستخدمه كـ index في المصفوفة: `pinAttempts[attempts]` .
وشرط الـ `while` هيبقى فيه `&&`

تحدّي إضافي: خلّي العمليات نفسها مصفوفة `["balance", "withdraw", "deposit"]` ونفّذهم كلهم ورا بعض بـ `for` .